

MiND: Minimal Deterministic Middleware for Auditable Large Language Model Interactions

Edervaldo José de Souza Melo¹

¹ Independent Researcher, Brazil

DOI: [10.xxxxxx/draft](https://doi.org/10.xxxxxx/draft)

Software

- [Review](#) ↗
- [Repository](#) ↗
- [Archive](#) ↗

Editor: ↗

Submitted: 09 September 2026

Published: unpublished

License

Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License ([CC BY 4.0](#)).

Summary

Large language models (LLMs) are increasingly embedded in research software, yet many integrations preserve only the visible prompt and answer while losing structured evidence about configuration, provider metadata, timing, failures, and the exact request procedure. MiND is an open-source, provider-neutral middleware that places a small audit boundary between an application and an LLM. For each attempted interaction it normalizes the input, constructs an explicit request, invokes a provider adapter, normalizes the result, and stores a versioned Cycle Artifact containing the observable execution context. The same core is available through a Python API, command-line interface, local HTTP API, and a self-contained Windows interface. MiND is deliberately non-agentic: it does not infer human intention, expose hidden model reasoning, or claim deterministic repetition of stochastic model output. Its purpose is narrower—to make the surrounding execution procedure inspectable and reusable.

Statement of need

Direct application-to-provider calls can leave research records fragmented across ad hoc logs, provider dashboards, and application-specific code. This complicates later inspection of what was submitted, which model and public parameters were selected, what metadata was returned, and whether an interaction succeeded or failed. These concerns are especially important because externally managed LLMs may change over time and stochastic outputs may differ even when the nominal prompt and model are unchanged. MiND therefore uses a bounded notion of reproducibility: it records the declared request procedure and observed result so that an execution can be inspected or attempted again, without claiming identical regeneration of remote model text. This approach is consistent with broader calls for transparent reporting and reproducible machine-learning practice ([Mitchell et al., 2019](#); [Pineau et al., 2021](#)).

MiND targets researchers and developers who need a lightweight, locally controlled provenance layer around LLM calls rather than a full agent framework. Typical uses include documenting LLM-assisted research steps, comparing providers under declared conditions, retaining queryable evidence for an application, and linking experimental records to the exact interaction artifacts that produced them.

State of the field

LLM observability systems such as MLflow Tracing ([MLflow Project, 2026](#)) and Langfuse ([Langfuse, 2026](#)) provide rich tracing, evaluation, prompt-management, and team-oriented monitoring capabilities. MiND does not attempt to replace these platforms. Its scholarly contribution is a deliberately smaller reference design centered on one explicit interaction boundary and one versioned provider-neutral artifact. The base package has no mandatory

39 runtime dependency, includes a deterministic offline mock, supports local JSON Lines and
40 SQLite storage, and can run without a hosted observability service.

41 This design choice favors inspectability, local ownership of research records, low-friction teaching
42 and prototyping, and integration into software where a researcher wants a compact audit object
43 rather than an operational monitoring platform. Existing tools are preferable when large-scale
44 dashboards, distributed tracing, evaluation pipelines, or collaborative lifecycle management are
45 required; MiND is intended for the narrower case in which the research workflow itself should
46 own a minimal and portable record of each LLM interaction.

47 Software design

48 MiND mediates one ordered path. An embedding application supplies text and an externally
49 selected configuration; MiND normalizes the input, builds system and user messages, calls the
50 configured provider, normalizes the provider result, persists a Cycle Artifact, and returns the
51 response with a `cycle_id`. Failed provider calls are also recorded with a safe structured error
52 before the failure is returned to the caller.

53 Providers implement a small protocol exposing a name, model, and generation method. Version
54 1.0.6 includes an offline deterministic mock plus optional OpenAI and Anthropic adapters.
55 Vendor-specific exceptions are converted to provider-neutral error records, and raw exceptions
56 are not intentionally copied into artifacts. Custom model services can be integrated by
57 implementing the same protocol.

58 The principal research object is the `mind.cycle/1` Cycle Artifact. It records identity and status,
59 UTC timestamps, duration, normalized input, public execution configuration, provider and
60 model metadata, normalized output, safe error information, and an extensions namespace for
61 future versioned metadata. The artifact records observable interaction evidence rather than
62 claims about latent reasoning or human intent.

63 Persistence is available through JSON Lines and SQLite. JSON Lines favors transparent append-
64 only inspection; SQLite provides indexed lookup, pagination, and transactional local persistence.
65 The same runtime can be reached through Python, CLI, and a versioned local HTTP API. A
66 Windows MSI packages the runtime and local control interface. Provider credentials entered
67 through the Windows application are stored in Windows Credential Manager and are not
68 intentionally returned by the settings API or written into Cycle Artifacts.

69 The software is tested across Python 3.9–3.13. Release 1.0.6 reports 75 automated tests and
70 86% branch-aware coverage, with additional linting, type checking, packaging checks, and
71 Windows install/upgrade/uninstall validation. The project is licensed under Apache-2.0 and
72 includes contribution, security, citation, changelog, and validation documentation.

73 Research impact statement

74 MiND has a public development history beginning in February 2026 and has progressed through
75 multiple tagged releases to version 1.0.6. The software is distributed as a Python package
76 and through public GitHub releases, with a version-specific archival record on Zenodo (DOI:
77 10.5281/zenodo.22291450) (Melo, 2026a). An earlier preprint introduced the initial MiND
78 concept and implementation (Melo, 2026b), creating a citable public record before the present
79 software-paper submission.

80 The current research significance is primarily methodological and infrastructural rather than
81 evidence of widespread external adoption. MiND provides a concrete, testable artifact model for
82 studies that need to document LLM execution conditions without depending on one provider's
83 conversation history or hosted observability stack. Its deterministic mock, provider-neutral
84 schema, local storage options, and documented integration boundary make it immediately

85 usable for reproducibility experiments, provider comparisons, teaching, and provenance-aware
86 research software. External adoption is not claimed at this stage; instead, the project offers
87 public releases, documentation, tests, contribution pathways, and stable interfaces intended to
88 support near-term reuse and independent evaluation.

89 AI usage disclosure

90 OpenAI ChatGPT was used during parts of the software-development, documentation,
91 and manuscript-preparation process. Assistance included code-generation and refactoring
92 suggestions, test scaffolding, documentation drafting, and language editing. Model versions
93 varied during the development period; GPT-5.6 Sol was used during final preparation of this
94 manuscript. The author made the architectural, methodological, and release decisions and
95 manually reviewed, edited, executed, and validated AI-assisted outputs. Software behavior was
96 checked through automated tests, continuous integration, packaging validation, and manual
97 interface testing. No generative model is listed as an author.

98 Acknowledgements

99 The author acknowledges the maintainers of the open-source Python, FastAPI, SQLite,
100 packaging, and testing ecosystems used to build and validate MiND.

101 Funding

102 This work received no specific grant from any funding agency in the public, commercial, or
103 not-for-profit sectors.

104 Langfuse. (2026). *Langfuse: Open source LLM engineering platform*. <https://langfuse.com/>.

105 Melo, E. J. de S. (2026a). *MiND v1.0.6* (Version 1.0.6). Zenodo. <https://doi.org/10.5281/zenodo.22291450>

107 Melo, E. J. de S. (2026b). *MiND: A minimal deterministic middleware for auditable LLM*
108 *interaction*. *Qeios*. <https://doi.org/10.32388/ZIT5ZI>

109 Mitchell, M., Wu, S., Zaldivar, A., Barnes, P., Vasserman, L., Hutchinson, B., Spitzer, E.,
110 Raji, I. D., & Gebru, T. (2019). Model cards for model reporting. *Proceedings of the*
111 *Conference on Fairness, Accountability, and Transparency*, 220–229. [https://doi.org/10.](https://doi.org/10.1145/3287560.3287596)
112 [1145/3287560.3287596](https://doi.org/10.1145/3287560.3287596)

113 MLflow Project. (2026). *MLflow Tracing*. <https://mlflow.org/docs/latest/genai/tracing/>.

114 Pineau, J., Vincent-Lamarre, P., Sinha, K., Larivière, V., Beygelzimer, A., d'Alché-Buc, F.,
115 Fox, E., & Larochelle, H. (2021). Improving reproducibility in machine learning research.
116 *Journal of Machine Learning Research*, 22(164), 1–20.